BizPlus AI Commerce Requirements

Transaction monitoring Test Mode commerce and approved live checks

BizPlus AI Team 13 October 2026

BizPlus AI combines a business-outcome monitoring demo with a six-product storefront using actual Razorpay Test Checkout and bounded HTTPS website checks. The complete React dashboard and Express API run on one public Render origin. This specification records the implemented behavior, verified evidence, and remaining storage limits for the team and hackathon judges.

Problem and business outcome

An HTTP success response can hide an incomplete customer journey. In the owned synthetic shop, checkout succeeds while a paused worker leaves order confirmation missing. A business assertion detects the failure despite HTTP 200. The operator inspects captured evidence, approves resuming the original worker, and verifies the original order without repeating checkout.

Implemented capabilities

Capability	Required outcome
Synthetic monitoring	Run healthy, checkout HTTP 500, and missing-confirmation scenarios. Show actual HTTP results, assertions, logs, fingerprints, and approved recovery.
Test Mode storefront	Choose from six server-priced products, create a provider order, complete Razorpay Test Checkout, and show a server-verified receipt.
Approved live checks	Call a server-configured owned HTTPS target and validate status, expected content, and response evidence.

Judge demonstration

First run the missing-confirmation scenario and recover its original order. Then open Storefront, buy the INR 599 Orbit Mini Speaker using Razorpay Test Mode, and inspect its paid receipt. Finally open Live tests and check the approved Render health target. Refresh to inspect retained records and show the mobile layout.

Rules-based diagnosis remains visibly labelled. No LLM API participates in these explanations. Browser session IDs isolate demo records rather than authenticate production users. Shop purchases and live checks have separate records and do not change synthetic workflow success metrics.

Behavior and approval rules

Subsystem	Implementation requirement
Synthetic journey	Actual HTTP login, inventory, cart, checkout, and confirmation. Store expected and actual outcomes, response status, duration, assertions, and relevant logs. Skip dependent steps after failure.
Bounded recovery	Require explicit resume_receipt_worker approval. Verify the original order and one simulated payment. Failed verification remains open. This action does not modify Razorpay purchases.
Payment verification	Price the cart on the server. Validate callback HMAC, original provider order, captured status, INR amount, and provider paid totals before marking a receipt paid.
Retry handling	Keep one checkout key and unique provider IDs. Preserve unpaid or cancelled orders. Reconcile a received delayed-capture callback without creating another provider order.
Live targets	Choose a configured target ID, never an arbitrary submitted URL. Use read-only HTTPS with public DNS pinning, same-origin redirects, no credentials, and bounded time and response size.

API and interfaces

Existing /api/runs, /api/incidents, and /api/metrics endpoints serve the synthetic monitor. New /api/shop/config and /api/shop/products expose Test Mode configuration and the catalog. POST /api/shop/orders creates a checkout, GET /api/shop/orders retrieves session history, and POST /api/shop/orders/:id/verify verifies the payment callback. Optional /api/shop/webhook supports signed payment.captured events.

GET /api/live/targets lists approved targets. POST /api/live/runs starts a target check and GET /api/live/runs/:id retrieves its result. Browser requests use X-Demo-Session. Shared TypeScript contracts define commerce orders, checkout verification, live assertions, and response evidence.

Architecture and current storage

Node 24 serves the built React frontend and Express API on the same Render origin. Razorpay Test Mode supplies the provider checkout and payment records. The API owns prices, signature verification, receipts, and all provider secrets. The active hosted database is SQLite with temporary history.

An optional private PostgreSQL primary adapter passed tests against a real local Windows PostgreSQL instance. Hosted Supabase activation remains pending a valid PostgreSQL connection URI. An HTTPS Supabase project address cannot serve as DATABASE_URL. The diagram in the pitch labels this optional branch separately from active SQLite.

Verification evidence

Local automated testing and hosted smoke checks are separate evidence sets. The hosted core replay deliberately injects failures; its success percentage is not an uptime or service-reliability estimate. Report individual timings as observed samples rather than production benchmarks.

Check and scope	Observed result
Expanded local automated suite	33 of 33 passed
Hosted synthetic core HTTP smoke	18 of 18 passed
Hosted healthy synthetic journey sample	1834 ms
Hosted synthetic recovery sample	237 ms with one simulated payment before and after
Razorpay Test Checkout	Public Render purchase of INR 599 captured and server verified. Paid receipt survived refresh.
Live Render health check	HTTP 200; 79 ms public browser sample and 215 ms earlier API replay
Earlier service wake-up check	HTTP 503 captured as an assertion failure
PostgreSQL primary and migration	Actual local Windows PostgreSQL 18 tests passed; hosted Supabase is unverified

Acceptance cases

Synthetic acceptance covers all three scenarios, evidence-linked rules, recurring failure grouping, approval, one active run per session, idempotent recovery, and unchanged original payments. Cross-session reads must fail, and verification failure must keep the incident open.

Commerce acceptance covers server-priced totals, tampered signatures, wrong order or amount, provider status, duplicate callbacks, delayed capture, and retained receipts. Test Mode transfers no real funds and ships no products. The storefront is separate from automatic synthetic remediation.

Live-check acceptance covers approved target selection, private-address blocking, pinned DNS, bounded redirects, timeouts, response limits, and readable evidence. These are HTTP checks rather than arbitrary browser login or checkout automation. Mobile screenshots verify responsive layout rather than a physical second-device network test.

Public release status

The complete Render application is live. Its hosted synthetic core passed 18 HTTP smoke checks. The public browser verified bounded recovery and retained its direct route on refresh. Public Razorpay Test Checkout produced a captured INR 599 payment and a server-verified receipt that survived refresh. Approved live checks returned actual HTTPS evidence. Hosted SQLite is temporary; Supabase cloud activation and a physical second-device test remain pending.

Public application https://bizplus-ai-api.onrender.com

Deployment and configuration

Render builds from repository root with npm ci and npm run build:backend, then starts with npm start. SERVE_FRONTEND=true also builds and serves frontend/dist. Pin Node 24.19.0, use PORT on 0.0.0.0, and configure /health. VITE_API_BASE_URL and CORS_ORIGIN use the exact Render public origin.

Keep Razorpay Test Mode secrets and any database credentials in server environment settings. The browser receives only the public test key ID. Live Razorpay keys are rejected. Set a valid postgresql:// or postgres:// DATABASE_URL only when enabling the optional private database, and retain certificate verification. A configured database failure stops startup rather than silently switching stores.

Storage and presentation limits

Render currently uses temporary SQLite. Paid Test Mode receipts and monitor records may disappear after a hosted restart, redeploy, or service sleep. The verified refresh behavior does not establish durable cloud storage. Use a valid managed PostgreSQL connection and verify migration before making that persistence claim.

The synthetic sandbox remains available as a local presentation fallback after installing dependencies. Actual Razorpay Checkout and live HTTPS checks require internet access. A previous Render wake-up returned HTTP 503; show the measured failure and subsequent successful check rather than promising instant availability.

Team ownership and six hour schedule

Owner	Primary work
You	Express runner, data adapters, owned sandbox, server payment verification.
Member 2	Dashboard, storefront, live-check UI, mobile layout, screenshots and pitch.
Member 3	Rules, fingerprints, approval tests, hosted verification, demo rehearsal.

Agree shared contracts in the first 30 minutes. Deploy a skeleton by minute 60. Connect core workflows during hours one to three and finish bounded recovery by hour four. Freeze features for verification and polish during hour five, then use the final hour for evidence, artifacts, and two rehearsals. Commerce and live checks extend the verified core rather than displacing it.

Roadmap and positioning

Next activate and verify durable managed storage, add authenticated operators and an action audit policy, then evaluate optional LLM summaries against captured evidence. Existing products already offer synthetic response assertions and AI diagnosis. This prototype makes the missing business outcome, bounded approval, and verification easy to demonstrate while connecting actual Test Mode commerce and owned HTTPS checks.

References

Razorpay checkout: https://razorpay.com/docs/payments/payment-gateway/web-integration/standard/integration-steps/

Supabase database connection: https://supabase.com/docs/guides/database/connecting-to-postgres

Render free service limits: https://render.com/docs/free

New Relic assertions: https://docs.newrelic.com/docs/synthetics/synthetic-monitoring/scripting-monitors/write-synthetic-api-tests/

Checkly diagnosis: https://www.checklyhq.com/docs/resolve/ai-root-cause-analysis/overview/